

Actividad 21-09-2026

PASO 0: Building y Configuración de Navegación

En esta captura se muestra la configuración del sistema de enrutamiento en el archivo de rutas principal. Se ha añadido el path 'galeria' apuntando al GaleriaComponent, lo que permite a la aplicación resolver la URL y renderizar nuestro nuevo componente correctamente en el navegador.

```
myApp > src > app > TS app.routes.ts > ...
1  import { Routes } from '@angular/router';
2  import { GaleriaComponent } from './galeria/galeria.component';
3
4  export const routes: Routes = [
5    {
6      path: 'home',
7      loadComponent: () => import('./home/home.page').then((m) => m.HomePage),
8    },
9    {
10     path: 'galeria',
11     loadComponent: () => import('./galeria/galeria.component').then((m) => m.GaleriaComponent),
12   },
13   {
14     path: '',
15     redirectTo: 'home',
16     pathMatch: 'full',
17   },
18 ]
19 {
20   path: 'galeria',
21   component: GaleriaComponent
22 }
```

PASO 1: Auditoría de entorno

Captura de la terminal integrada tras ejecutar los comandos de auditoría del sistema. Se verifica que el entorno de desarrollo está correctamente configurado con Node.js en su versión 24.21.0, Ionic CLI en la versión 7.2.1 y Angular CLI en la versión 22.0.1.

```
PS C:\Users\alvar\OneDrive\Documentos\Nebrija\2º DAM\Desarrollo de Interfaces\Proyectos\VS Code\myApp> node -v
v24.21.0
```

```
PS C:\Users\alvar\OneDrive\Documentos\Nebrija\2º DAM\Desarrollo de Interfaces\Proyectos\VS Code\myApp> ionic -v
7.2.1
```

```
PS C:\Users\alvar\OneDrive\Documentos\Nebrija\2º DAM\Desarrollo de Interfaces\Proyectos\VS Code\myApp> ng version
```



```
Angular CLI      : 22.0.1
Angular          : 22.0.1
Node.js          : 24.21.0
Package Manager  : npm 11.19.0
Operating System : win32 x64
```

Package	Installed Version	Requested Version
@angular/build	22.0.1	22.0.1
@angular/cli	22.0.1	22.0.1
@angular/common	22.0.1	22.0.1
@angular/compiler	22.0.1	22.0.1
@angular/compiler-cli	22.0.1	22.0.1
@angular/core	22.0.1	22.0.1
@angular/forms	22.0.1	22.0.1
@angular/language-service	22.0.1	22.0.1
@angular/platform-browser	22.0.1	22.0.1
@angular/router	22.0.1	22.0.1
rxjs	7.8.2	~7.8.0
typescript	6.0.3	~6.0.0
vitest	4.1.11	^4.0.8

PASO 2: Análisis de dependencias

Fragmento del archivo package.json. Se constata que el proyecto utiliza la versión 22.0.1 del framework (@angular/core). Se aprecian claramente los dos bloques estructurales: las dependencies, que definen las librerías críticas para el funcionamiento de la app en producción, y las devDependencies, que contienen herramientas de soporte exclusivas para la etapa de desarrollo (como Angular CLI o TypeScript) que no se incluirán en el bundle final.

```
15  ✓  "dependencies": {
16      "@angular/common": "22.0.1",
17      "@angular/compiler": "22.0.1",
18      "@angular/core": "22.0.1",
19      "@angular/forms": "22.0.1",
20      "@angular/platform-browser": "22.0.1",
21      "@angular/router": "22.0.1",
22      "@capacitor/app": "8.1.1",
23      "@capacitor/core": "8.5.2",
24      "@capacitor/haptics": "8.0.2",
25      "@capacitor/keyboard": "8.0.5",
26      "@capacitor/status-bar": "8.0.3",
27      "@ionic/angular": "^9.0.0",
28      "ionicons": "^8.1.0",
29      "rxjs": "~7.8.0",
30      "tslib": "^2.3.0"
31  },
32  ✓  "devDependencies": {
33      "@angular/build": "22.0.1",
34      "@angular/cli": "22.0.1",
35      "@angular/compiler-cli": "22.0.1",
36      "@angular/language-service": "22.0.1",
37      "@capacitor/cli": "8.5.2",
38      "@ionic/angular-toolkit": "^13.0.0",
39      "angular-eslint": "22.0.0",
40      "eslint": "^9.16.0",
41      "jsdom": "^26.0.0",
42      "typescript": "~6.0.0",
43      "typescript-eslint": "^8.61.0",
44      "vitest": "^4.0.8"
45  },
```

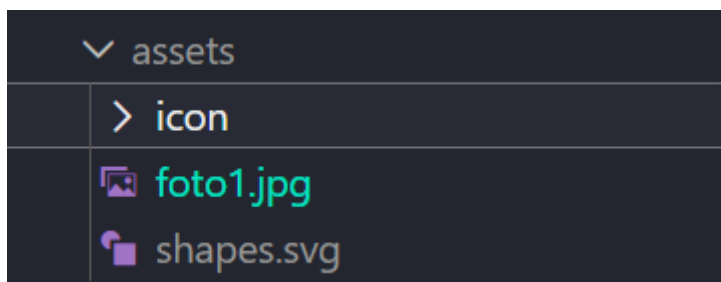
PASO 3: Comando ionic serve

Salida de la consola indicando el arranque exitoso del servidor web local mediante el comando ionic serve. El proyecto se ha desplegado en <http://localhost:8100>. Este proceso activa la utilidad técnica "Live Reload" (o HMR), la cual mantiene un socket abierto con el navegador para inyectar automáticamente cualquier cambio guardado en el código, permitiendo visualizar las modificaciones en tiempo real sin recargar la página completa ni perder el estado actual.

```
[INFO] Development server running!  
  
Local: http://localhost:8100  
  
Use Ctrl+C to quit this process  
  
[ng] → Local: http://localhost:8100/  
[ng] → press h + enter to show help  
[INFO] Browser window opened to http://localhost:8100!
```

PASO 4: Gestión de Assets

Árbol de directorios del proyecto en Visual Studio Code. Se demuestra la correcta gestión de los recursos estáticos al haber alojado la imagen de prueba (foto1.jpg) estrictamente dentro de la ruta src/assets/. Durante la compilación, el builder mapea esta carpeta, lo que nos permite referenciar la imagen más adelante mediante rutas relativas simples.



PASO 5: Desarrollo del Componente galería

Código fuente de los archivos del componente que demuestran la arquitectura Modelo-Vista. En el controlador TypeScript (galeria.component.ts), se ha aplicado un tipado estricto para inicializar las variables de estado y se ha implementado el CUSTOM_ELEMENTS_SCHEMA para que Angular procese nativamente las etiquetas Standalone de Ionic. En la vista (galeria.component.html), se establece una comunicación unidireccional (One-Way Data Binding) utilizando Interpolación {{ }} para renderizar dinámicamente el texto del encabezado, y Property Binding [src] para inyectar la ruta local en el atributo de la etiqueta imagen.

```
myApp > src > app > galeria > TS galeria.component.ts > ...
1  import { Component, OnInit, CUSTOM_ELEMENTS_SCHEMA } from '@angular/core';
2
3  @Component({
4    selector: 'app-galeria',
5    templateUrl: './galeria.component.html',
6    styleUrls: ['./galeria.component.scss'],
7    imports: [],
8    schemas: [CUSTOM_ELEMENTS_SCHEMA]
9  })
10 export class GaleriaComponent implements OnInit {
11
12   public tituloSeccion: string = 'Galería de Imágenes Dinámica';
13   public rutaImagenLocal: string = 'assets/foto1.jpg';
14
15   constructor() { }
16
17   ngOnInit() {}
18
19 }
20
```

```
myApp > src > app > galeria > <> galeria.component.html > ...
1  <ion-header>
2    <ion-toolbar>
3      <ion-title>{{ tituloSeccion }}</ion-title>
4    </ion-toolbar>
5  </ion-header>
6
7  <ion-content class="ion-padding">
8    <div class="image-container">
9      <img [src]="rutaImagenLocal" alt="Imagen cargada localmente" />
10    </div>
11  </ion-content>
12
```

PASO 6: Validación Final

Validación visual desde el navegador web accediendo a la ruta configurada. La captura confirma que la comunicación de datos fluye correctamente desde la lógica a la vista: la interpolación ha impreso el título "Galería de Imágenes Dinámica" en el componente `<ion-title>`, y el *Property Binding* ha resuelto con éxito la carga del *asset* estático, renderizando la imagen esperada en el cuerpo de la aplicación.

